



Testes de Unidade & Qualidade

Dominando JUnit 5 e Mock Objects para um Código Resiliente



APRESENTADOR
Lucas G. Nadalete

Instituição: **FATEC/SJC** • Data: **Março 2026**

Testes de Unidade garantem a integridade da menor parte do código.

Uma técnica de teste de software onde unidades individuais de código (métodos ou funções) são isoladas e validadas de forma independente.



Detecção Precoce

Identificação de falhas no início do ciclo de desenvolvimento, reduzindo custos de correção.



Refatoração Segura

Liberdade para alterar o código sem medo de quebrar funcionalidades existentes.



Documentação Viva

O teste atua como um exemplo prático e sempre atualizado de como usá-lo

“

**"Se você não pode
testar, você não pode
confiar."**

De Smalltalk para o Mundo: A Revolução de Kent Beck.

1989

A Origem (SUnit)

Kent Beck criou o **SUnit** para Smalltalk, introduzindo o conceito pioneiro de framework de teste automatizado.

1997

O Voo do JUnit

Durante um voo de Zurique para Atlanta, **Kent Beck** e **Erich Gamma** portaram o SUnit para Java, criando o lendário **JUnit**.



Legado xUnit: Essa arquitetura inspirou frameworks em quase todas as linguagens modernas, como NUnit (C#), PyTest (Python) e PHPUnit. A filosofia foca em **automação total** e **feedback imediato** para o desenvolvedor.

JUnit 5: Uma Estrutura Modular e Flexível.



JUnit Platform

A base da infraestrutura . Define a interface `TestEngine` para lançar frameworks de teste na JVM, permitindo que IDEs e ferramentas de build rodem testes de forma unificada.



JUnit Jupiter

A nova API de programação. Contém as anotações modernas e o motor de execução para testes escritos no padrão do JUnit 5, aproveitando recursos do Java 8+.



JUnit Vintage

O componente de **retrocompatibilidade**. Fornece um motor para rodar testes baseados em JUnit 3 e 4, permitindo uma migração gradual sem quebrar o código legado.



Essa modularidade permite o **desacoplamento** entre o framework de teste e as ferramentas de execução, facilitando a evolução contínua do ecossistema Java.

Controlando o Ciclo de Vida com Anotações Essenciais.

@BeforeEach

Executa o método antes de **cada** teste. Ideal para preparar o cenário e instanciar objetos.

@AfterEach

Executa após **cada** teste. Útil para limpar dados ou resetar estados.

@Test

Define o método como uma unidade de teste a ser executada pelo framework.

@BeforeAll / @AfterAll

Executam uma única vez para a classe toda (devem ser métodos estáticos).

```
class CalculadoraTest {  
  
    @BeforeEach  
    void setup() {  
        // Preparação antes de CADA teste  
    }  
  
    @Test  
    void deveSomarDoisNumeros() {  
        assertEquals(4, 2 + 2);  
    }  
  
    @AfterEach  
    void tearDown() {  
        // Limpeza após CADA teste  
    }  
}
```

@ParameterizedTest: Executando o mesmo teste com múltiplos dados.

Eficiência e Reuso

Permite rodar um único método de teste várias vezes com diferentes argumentos, reduzindo drasticamente a duplicação de código.

Fontes Flexíveis

Suporta diversas fontes de dados como @ValueSource, @CsvSource, @MethodSource e @EnumSource.

Legibilidade

Facilita a visualização de diferentes cenários de entrada e saída em uma única estrutura lógica.

```
@ParameterizedTest
@ValueSource(ints = {1, 2, 3})
void deveSerPositivo(int numero) {
    assertTrue(numero > 0);
}

@ParameterizedTest
@CsvSource({
    "1, 1, 2",
    "2, 3, 5",
    "10, 5, 15"
})
void deveSomarValores(int a, int b, int res) {
    assertEquals(res, calc.somar(a, b));
}
```

Diferenciando Mocks, Stubs e Fakes.



Stub

Fornecer **respostas prontas** para chamadas feitas durante o teste. Foca no estado retornado.

"Sempre retorne 'true' quando perguntado se o usuário é admin."



Mock

Focado em **verificação de comportamento**. Garante que um método foi chamado com parâmetros específicos.

"Verifique se o método 'enviarEmail' foi chamado exatamente uma vez."



Fake

Possui uma **implementação funcional**, mas simplificada e inadequada para produção.

"Um banco de dados H2 em memória em vez de um PostgreSQL real."



Embora frameworks como o Mockito facilitem a criação de **Mocks** e **Stubs**, a escolha depende do que você deseja validar: o **resultado** (Stub) ou a **interação** (Mock).

Mock Objects: Simulando comportamentos para isolar dependências.

A Necessidade do Isolamento

Testes de unidade devem ser **rápidos e independentes** de fatores externos como Banco de Dados, APIs de terceiros ou sistemas de arquivos.

Substituímos a dependência real por um **objeto "dublê"** que retorna valores pré-definidos e controlados pelo teste.



Velocidade

Execução instantânea sem latência de rede ou acesso a disco.



Determinismo

Comportamento previsível, independente do estado de sistemas externos.



Foco Total

Garante que o teste valide apenas a lógica da unidade em questão.

Mockito + JUnit 5: A combinação perfeita para testes robustos.

O Mockito é o framework de mocking padrão da indústria para Java, permitindo a criação de objetos simulados de forma simples e expressiva.



mock()

Criação dinâmica do objeto simulado a partir de uma classe ou interface.



when(...).thenReturn(...)

Definição de expectativas: o que o mock deve retornar ao ser chamado.



verify(...)

Garante que métodos específicos foram invocados com os parâmetros corretos.

Integração Nativa

O Mockito se integra perfeitamente ao JUnit 5 através de extensões, automatizando a inicialização de mocks e injeção de dependências.

```
@ExtendWith(MockitoExtension.class)
class MeuTeste { ... }
```

Elimina a necessidade de chamadas manuais para inicializar os mocks no setup.

Integrando Lógica de Negócio e Mocks com Mockito.

Cenário de Negócio

Testando um `PedidoService` que depende de um `EstoqueRepository` para validar a disponibilidade de produtos.

@Mock

Cria o objeto simulado do repositório, evitando acesso real ao banco de dados.

@InjectMocks

Instancia o serviço e injeta automaticamente os mocks criados nas suas dependências.

```
@ExtendWith(MockitoExtension.class)
class PedidoServiceTest {
    @Mock EstoqueRepository repository;
    @InjectMocks PedidoService service;

    @Test
    void deveFinalizarPedidoSeHouverEstoque() {
        // 1. Definir comportamento

        when(repository.temEstoque(1)).thenReturn(true);

        // 2. Executar ação

        boolean resultado = service.finalizar(1);

        // 3. Validar resultados

        assertTrue(resultado);
        verify(repository).baixarEstoque(1);
    }
}
```

Construindo uma Cultura de Qualidade através de Testes.

★ Princípios FIRST

F

Fast (Rápido)

Testes devem ser executados em segundos.

I

Independent (Independente)

Um teste não deve depender de outro.

R

Repeatable (Repetível)

Resultados consistentes em qualquer ambiente.

S

Self-validating (Auto-validável)

O teste deve ter um resultado claro: Passa ou Falha.

T

Timely (Oportuno)

Escritos no momento certo (idealmente antes do código).

☑ Checklist de Sucesso

- ✓ Mantenha os testes simples e legíveis para toda a equipe.
- ✓ Evite lógica complexa dentro dos próprios testes.
- ✓ Teste comportamentos e regras de negócio, não implementações.

Próximos Passos:

Explore o **TDD (Test Driven Development)** para guiar o design do seu software desde o primeiro dia.